

Agentic Architect Challenge - Architecture Summary

1. Overall Architecture

- **Part 1:** Email -> critical check -> classify -> knowledge base check -> safe response or human escalation
- **Part 2:** URL -> fetch HTML -> remove irrelevant tags -> extract main content -> 8,000 character limit -> extractive summary -> 120 word limit
- **Part 3:** User -> LangGraph state -> decide route -> support guide / memory / calculator -> answer

2. Key Decisions and Trade-offs

Area	Decision	Trade-off
Critical support cases	Check data loss, outages, security issues, and >3 contacts in 7 days before response generation.	Safer, but more cases may reach humans.
Knowledge grounding	Use local FAQ/support guide and escalate when information is insufficient.	Less automation, lower hallucination risk.
Web scraping	Remove navigation/header/footer/script/style, prefer main/article/body, use timeout and HTTP checks.	Simple and lightweight, but not a full browser for JavaScript-heavy sites.
Summary guardrail	Limit scraped input to 8,000 characters and output to 120 words.	Concise and predictable, but aggressive limits can remove detail.
Part 3 agent	LangGraph with in-memory checkpointing, document routing, memory, and calculator.	Works locally without an external API; current prototype is deterministic rather than LLM-generated.

3. Failure Points and Handling

Part 1 escalates unsafe or unsupported requests instead of guessing. Part 2 handles request failures, uses a 10-second timeout, and limits content size. Part 3 catches runtime errors at the CLI and uses a thread ID with InMemorySaver so conversation state can be retained during a run.

4. Testing and Observability

The repository contains automated tests for routing, scraping, content limits, summary limits, document answers, unknown information, calculator behavior,

and conversation memory. Run all tests with: `python -m unittest discover -s tests -v`. For production, add structured logs, metrics, trace IDs, retries, and an audit record for agent decisions.

5. Future Production Improvement

The main next step is to replace the deterministic Part 3 decision layer with an LLM adapter while retaining the existing document grounding, tool boundary, memory, and safety checks. The external Gemini/OpenAI credentials used during development were unavailable, so the submitted prototype keeps this dependency optional rather than embedding a non-working credential path.

6. Run Locally

```
pip install langgraph requests beautifulsoup4 | python part1/routing.py | python part2/scrapper.py |  
python part3/agent.py (no API key required)
```